

CHƯƠNG IV – LẬP TRÌNH ĐỒ HỌA

LẬP TRÌNH NÂNG CAO

Giảng viên: Lê Văn Hùng

 0906238311

 hunglv@hvnh.edu.vn

1. Thêm thư viện đồ họa vào Dev C

❖ *Đường dẫn (Path):*

+ Với Windows 32 bit: Path="C:\Program Files\Dev-Cpp"
+ Với Windows 64 bit: Path="C:\Program Files
(x86)\Embarcadero\Dev-Cpp\"

1. Cài đặt thư viện đồ họa trong windows 32 bit

❖ +Đối với Dev C++ 32 bit:

Bước 1: Copy 2 file “graphics.h” và “winbgim.h” vào thư mục
(Path)\MinGV32\include”.

Bước 2: Copy file “libbgi.a” vào thư mục “(Path)\MinGV32\lib”.

Bước 3: Copy 2 file “6-ConsoleAppGraphics.template” và
“ConsoleApp_cpp_graph.txt” vào thư mục “(Path)\Templates”.

1. Cài đặt thư viện đồ họa trong windows 64 bit

❖ +Đối với Dev C++ 64 bit:

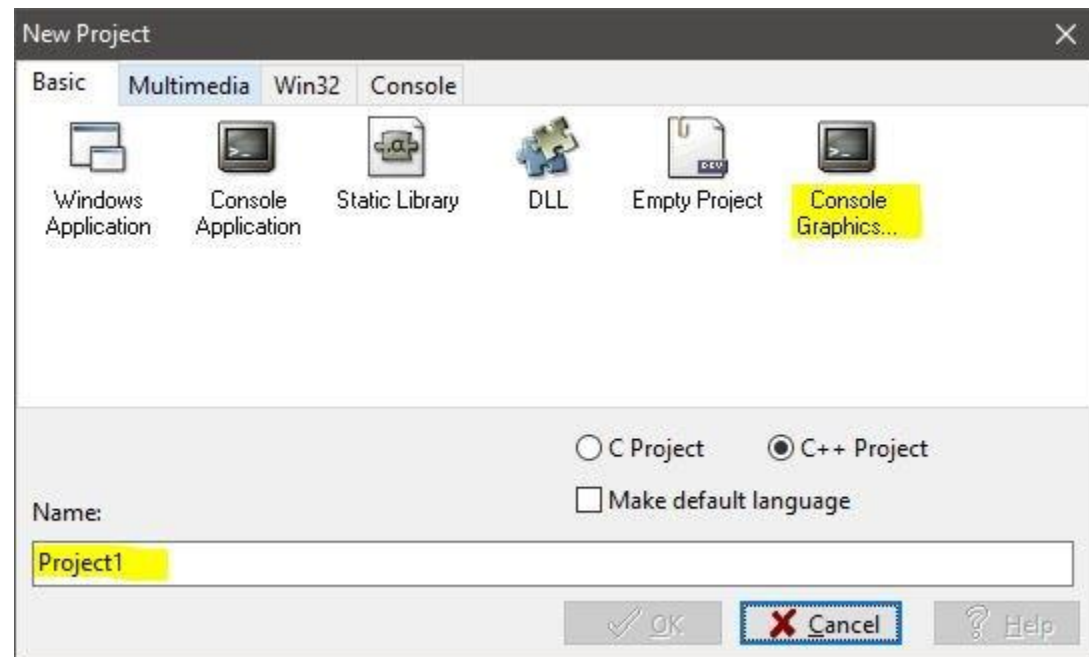
Bước 1: Copy 2 file “graphics.h” và “winbgim.h” vào thư mục C:\Program Files (x86)\Embarcadero\Dev-Cpp\TDM-GCC-64\include”.

Bước 2: Copy file “libbgi.a” vào thư mục “C:\Program Files (x86)\Embarcadero\Dev-Cpp\TDM-GCC-64\lib”.

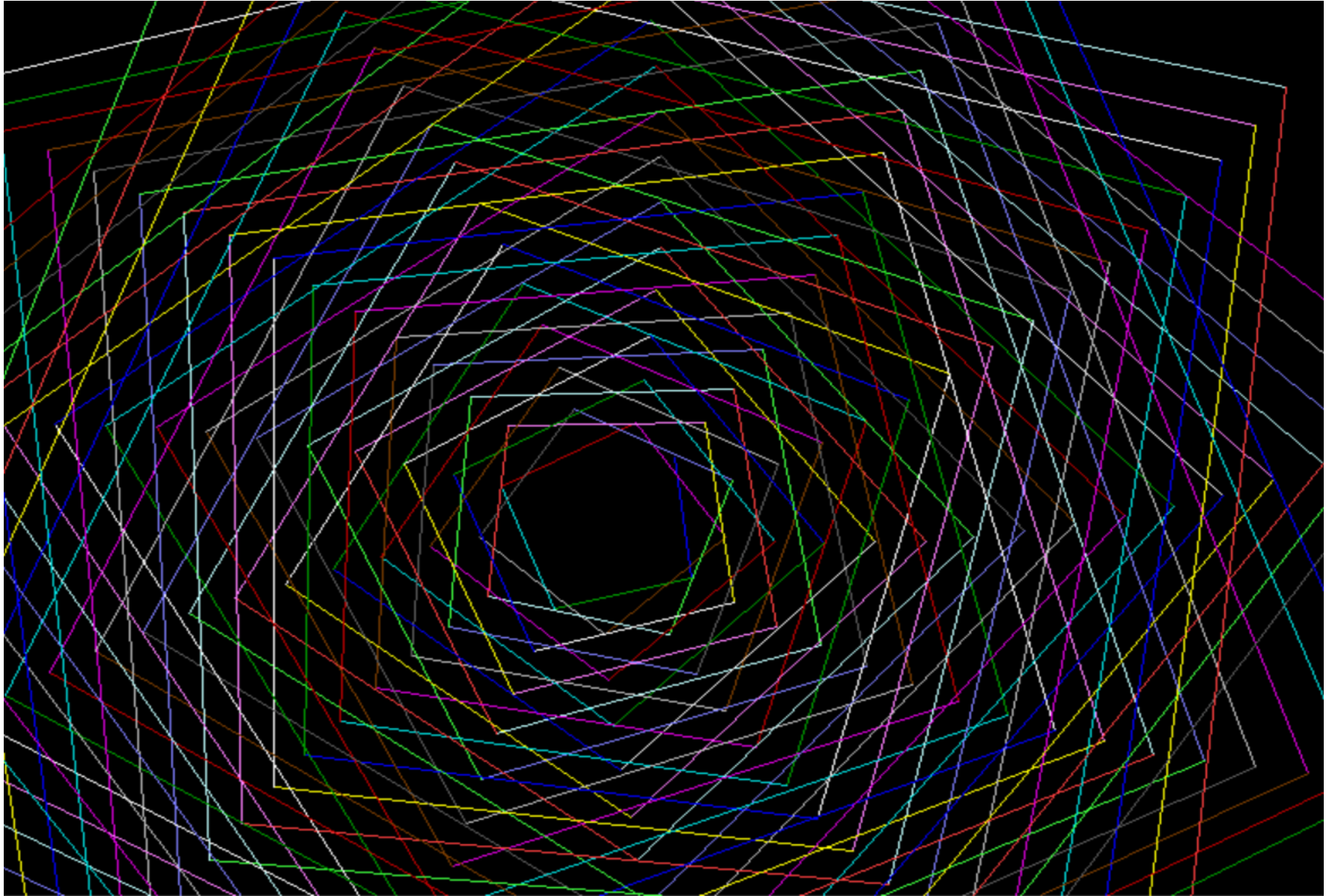
Bước 3: Copy 2 file “6-ConsoleAppGraphics.template” và “ConsoleApp_cpp_graph.txt” vào thư mục “(Path)\Templates”.

- ❖ Bước 1: Nếu cài Dev C++ 32 bit thì bỏ qua bước này, còn nếu là 64 bit thì các em chọn trình biên dịch là **TDM-GCC 32 bit** ứng với phiên bản Dev C++ bạn cài đặt (*Đây là bước bắt buộc và phải làm từ đầu, nếu không thì sẽ không chạy được đồ họa*).

- ❖ *Bước 2:* Tạo một Project mới bằng cách chọn File -> New -> Project...
- + Chọn Console Graphics Application.
 - + Tick vào ô C++ Project.
 - + Gõ tên Project vào ô Name.
 - + Click Ok và chọn vị trí lưu



Chạy thử chương trình độ họa vẽ hình xoắn ốc



Khái niệm đồ họa

- ❖ Chế độ đồ họa màn hình được chia thành điểm ảnh (pixel). Màn hình VGA thông thường độ phân giải 480 hàng và 640 cột. Góc trên bên trái tọa độ là $(x,y)=(0,0)$, góc dưới phải tọa độ là $(x,y)=(639,479)$. Trục x hướng sang phải, trục y hướng xuống dưới.
- ❖ Khi chạy chương trình đồ họa ta cần có tệp Egavga.bgi, các tệp Goth chr, litt chr, sans chr, trip chr (*chứa các font chữ*). Thông thường chúng ta sử dụng đồ họa ở chế độ 16 màu.



Khởi tạo chế độ đồ họa

❖ Hàm `void initgraph(int *graphdriver, int *graphmode, char *s)` dùng để khởi động chế độ đồ họa, trong đó `s` là đường dẫn tới tệp `egavga.bgi`, nếu `s=""` (rỗng) thì máy tự tìm tệp `egavga.bgi` trên thư mục chủ.

graphdriver DETECT (0)	graphmode	Độ phân giải
CGA (1)	CGAC0 (0)CGAC1 (1)CGAC2 (2)CGAC3 (3)CGAHi (4)	320x200320x200320x200320x200640x200
MCGA (2)	MCGA0 (0)MCGA1 (1)MCGA2 (2)MCGA3 (3)MCGAMed (4)MCGAHi (5)	320x200320x200320x200320x200640x200640x480
EGA (3)	EGAL0 (0)EGAHi (1)	640x200640x350
EGA64 (4)	EGA64LO (0)EGA64Hi (1)	640x200640x350
EGAMONO (5)	EGAMONOH (0)	640x350
VGA (9)	VGALO (0)VGAMED (1)VGAHi (2)	640x200640x350640x480
HERCMONO (7)	HERCMONOH	720x348
ATT400 (8)	ATT400C0 (0)ATT400C1 (1)ATT400C2 (2)ATT400C3 (3)ATT400MED (4)ATT400HI (5)	320x200320x200320x200320x200640x400640x400
PC3270 (10)	PC3270HI (0)	720x350
IBM8514 (6)	PC3270LO (0)PC3270HI (1)	640x480 256 màu1024x768 256 màu

- ❖ Giá trị của **graphdriver** và **graphmode** được xác định trong bảng sau:
 - + Nếu **graphdriver = VGA (hay 9)** thì giá trị của **graphmode** có thể là **VGALO (hay 0)**. Ứng với độ phân giải màn hình 640 x 200 điểm ảnh, **graphdriver=VGAHI (hay 2)** ứng với độ phân giải 640 x 480 điểm ảnh.
 - + Nếu **graphdriver=DETECT (hay 0)** thì chương trình tự tìm kiểu màn hình đang dùng và gán **graphdriver**.

- Bảng trên còn cho thấy độ phân giải còn phụ thuộc cả vào màn hình và mode. Ví dụ như trong màn hình EGA nếu dùng EGALo thì độ phân giải là 640x200 (Hàm getmaxx() cho giá trị cực đại của số điểm theo chiều ngang của màn hình. Với màn hình EGA trên : 639, Hàm getmaxy() cho giá trị cực đại của số điểm theo chiều dọc của màn hình. Với màn hình EGA trên : 199).
- Nếu không biết chính xác kiểu màn hình đang sử dụng thì ta gán cho biến graphdriver bằng DETECT hay giá trị 0. Khi đó, kết quả của initgraph sẽ là **kiểu màn hình đang sử dụng được phát hiện, giá trị của nó được gán cho biến graphdriver.**

Khởi tạo đồ họa

- ❖ Hàm **int graphresult(void)** cho mã lỗi khi khởi động đồ họa. Hàm cho giá trị bằng 0 tức là không có lỗi, cho giá trị khác không (mã lỗi) thì có lỗi.
- ❖ Hàm **void closegraph(void)** dùng để đóng chế độ đồ họa.

2. Các hàm đồ họa thông dụng trong C

Mẫu và màu nền

Đặt màu nền

Để đặt màu cho nền ta dùng thủ tục sau :

```
void setbkcolor(int màu);
```

Đặt màu đường vẽ

Để đặt màu vẽ đường ta dùng thủ tục sau :

```
void setcolor(int màu);
```

Đặt mẫu (kiểu) tô và màu tô

Để đặt mẫu (kiểu) tô và màu tô ta dùng thủ tục sau :

```
void setfillstyle(int mẫu, int màu);
```

2. Các hàm đồ họa thông dụng trong C

Bảng màu

Tên hằng	Giá trị số	Màu hiển thị
BLACK	0	Đen
BLUE	1	Xanh da trời
GREEN	2	Xanh lá cây
CYAN	3	Xanh lơ
RED	4	Đỏ
MAGENTA	5	Tím
BROWN	6	Nâu
LIGHTGRAY	7	Xám nhạt
DARKGRAY	8	Xám đậm
LIGHTBLUE	9	Xanh xa trời nhạt

LIGHTGREEN	10	Xanh lá cây nhạt
LIGHTCYAN	11	Xanh lơ nhạt
LIGHTRED	12	Đỏ nhạt
LIGHTMAGENTA	13	Tím nhạt
YELLOW	14	Vàng
WHITE	16	Trắng

2. Các hàm đồ họa thông dụng trong C

Bảng mẫu

Tên hằng	Giá trị số	Kiểu mẫu tô
EMPTY_FILL	0	Tô bằng màu nền
SOLID_FILL	1	Tô bằng đường liền nét
LINE_FILL	2	Tô bằng đường - - - - -
LTSLASH_FILL	3	Tô bằng ///
SLASH_FILL	4	Tô bằng /// in đậm
BKSLASH_FILL	5	Tô bằng \\ in đậm
LTBKSLASH_FILL	6	Tô bằng \\
HATCH_FILL	7	Tô bằng đường gạch bóng nhật

XHATCH_FILL	8	Tô bằng đường gạch bóng chữ thập
INTERLEAVE_FILL	9	Tô bằng đường đứt quãng
WIDE_DOT_FILL	10	Tô bằng dấu chấm thưa
CLOSE_DOT_FILL	11	Tô bằng dấu chấm mau

2. Các hàm đồ họa thông dụng trong C

Mẫu và màu nền:

- Để thay đổi giải màu đã được định nghĩa trong bảng trên, ta sử dụng hàm : **void setpalette**(int **số_thứ_tự_màu**, int **màu**);

chẳng hạn: `setpalette(0,lightcyan);`

- Hàm **int getmaxx(void)**: cho tọa độ màn hình x lớn nhất của kiểu màn hình đang dùng.

- Hàm **int getmaxy(void)**: cho tọa độ màn hình y lớn nhất của kiểu màn hình đang dùng.

- Hàm **int getmaxcolor(void)**: cho giá trị màu lớn nhất đang dùng.

2. Các hàm đồ họa thông dụng trong C

- ❖ Hàm `void setbkcolor(int color)`: đặt màu nền, màu nền ngầm định ngay sau khi khởi động đồ họa sẽ là màu đen BLACK (0).
- ❖ Hàm `int getbkcolor(void)`: lấy màu nền hiện tại.
- ❖ Hàm `void setcolor(int color )`: đặt màu nét vẽ. Màu ngầm định ngay khi khởi động là WHITE (15).
- ❖ Hàm `int getcolor(void)`: lấy màu vẽ hiện tại.
- ❖ Hàm `void cleardevice(void)`: xóa toàn bộ màn hình đồ họa (chức năng tương tự `clrscr()` trong chế độ mode văn bản).
- ❖ Hàm `void restorecrtmode (void)`: khôi phục lại chế độ màn hình như trước khi khởi động đồ họa.

2. Các hàm đồ họa thông dụng trong C

- ❖ Hàm int **getgraphmode**(void): lấy kiểu màn hình đồ họa hiện tại.
- ❖ Hàm void **setgraphmode**(int mode): lựa chọn kiểu đồ họa khác với kiểu ngầm định đặt bởi initgraph, xoá màn hình.
- ❖ Hàm **moveto**(int x,int y): di chuyển con trỏ vẽ tới toạ độ (x,y) trên màn hình.
- ❖ Hàm int **getx**(void): cho toạ độ x của con trỏ vẽ hiện tại.
- ❖ Hàm int **gety**(void): cho toạ độ y của con trỏ vẽ hiện tại.

Hàm vẽ điểm

Hàm :

```
void putpixel(int x, int y, int color);
```

sẽ tô điểm (x,y) theo màu xác định bởi **color**.

Hàm

```
unsigned getpixel(int x, int y);
```

sẽ trả về số hiệu màu của điểm ảnh ở vị trí (x,y).

Nếu điểm này chưa được tô màu bởi các hàm vẽ hoặc hàm putpixel (mà chỉ mới được tạo màu nền bởi setbkcolor) thì hàm cho giá trị 0.

Hàm vẽ đường tròn và cung tròn

Đường tròn: Để vẽ đường tròn ta dùng hàm :

void circle(int x, int y, int r);

Trong đó:

(x,y) là toạ độ tâm cung tròn.

r là bán kính đường tròn.

Ví dụ: Vẽ một đường tròn có tâm tại (100,50) và bán kính 30.

circle(100,50,30);

Hàm vẽ đường tròn và cung tròn

Cung tròn: Để vẽ một cung tròn ta dùng hàm:

void arc(int x, int y, int gd, int gc, int r);

Trong đó:

(x,y) là toạ độ tâm cung tròn.

gd là góc đầu cung tròn (0 đến 360 độ).

gc là góc cuối cung tròn (từ gd đến 360 độ).

r là bán kính cung tròn .

Ví dụ : Vẽ một cung tròn có tâm tại (100,50), góc đầu là 0, góc cuối là 180, bán kính 30.

arc(100,50,0,180,30);

Hàm vẽ đường tròn và cung tròn

Cung tròn: Để vẽ một cung tròn ta dùng hàm:

void arc(int x, int y, int gd, int gc, int r);

Trong đó:

(x,y) là toạ độ tâm cung tròn.

gd là góc đầu cung tròn (0 đến 360 độ).

gc là góc cuối cung tròn (từ gd đến 360 độ).

r là bán kính cung tròn .

Ví dụ : Vẽ một cung tròn có tâm tại (100,50), góc đầu là 0, góc cuối là 180, bán kính 30.

arc(100,50,0,180,30);

Hàm vẽ đường tròn và cung tròn

Cung elip

Để vẽ một cung elip ta dùng hàm :

```
void ellipse(int x, int y, int gd, int gc, int xr, int yr);
```

Trong đó :

(x,y) là toạ độ tâm cung elip.

gd là góc đầu cung tròn(0 đến 360 độ).

gc là góc cuối cung tròn (gd đến 360 độ).

xr là bán trục nằm ngang.

yr là bán trục thẳng đứng.

Ví dụ :

Vẽ một cung elip có tâm tại (100,50), góc đầu là 0, góc cuối là 180, bán trục ngang 30, bán trục đứng là 20.

```
ellipse(100,50,0,180,30,20);
```

Hàm vẽ đường tròn và cung tròn

Hình quạt

Để vẽ và tô màu một hình quạt ta dùng hàm :

```
void pieslice(int x, int y, int gd, int gc, int r);
```

Trong đó :

(x,y) là toạ độ tâm hình quạt.

gd là góc đầu hình quạt (0 đến 360 độ).

gc là góc cuối hình quạt (gd đến 360 độ).

r là bán kính hình quạt .

Ví dụ 1

- ❖ Chương trình dưới đây sẽ vẽ một cung tròn ở góc phần tư thứ nhất, một đường tròn và một hình quạt quét từ 90 đến 360 độ.

```
# include "graphics.h"
#include "stdio.h"
#include "conio.h"
main()
{
    int md=0,mode;
    initgraph(&md,&mode,"");
    setbkcolor(BLUE);
    setcolor(YELLOW);
    setfillstyle(SOLID_FILL,RED);;
    arc(160,50,0,90,45); // Vẽ cung tròn ở góc phần tư thứ nhất
    circle(160,150,45); // vẽ đường tròn
    pieslice(480,150,90,360,45); // Vẽ hình quạt từ 90 tới 360 độ
    getch();
    closegraph();
}
```

Hàm vẽ đường thẳng

Để vẽ đường thẳng nối hai điểm bất kỳ có tọa độ (x_1, y_1) và (x_2, y_2) ta sử dụng hàm sau :

```
void line(int x1, int y1, int x2, int y2);
```

Con chạy đồ hoạ giữ nguyên vị trí.

Để vẽ đường thẳng nối từ điểm con chạy đồ hoạ đến một điểm bất kỳ có tọa độ (x, y) ta sử dụng hàm sau :

```
void lineto(int x, int y);
```

Con chạy sẽ chuyển đến vị trí (x, y) .

Để vẽ một đường thẳng từ vị trí con chạy hiện tại (giả sử là điểm x, y) đến điểm có tọa độ $(x+dx, y+dy)$ ta sử dụng hàm sau :

```
void linerel(int dx, int dy);
```

Con chạy sẽ chuyển đến vị trí $(x+dx, y+dy)$.

Hàm vẽ đường thẳng

Để di chuyển con chạy đến vị trí (x,y), ta sử dụng hàm sau :

```
void moveto(int x, int y);
```

Chọn kiểu đường

```
Hàm void setlinestyle(int kiểu_đường, int mẫu, int độ_dày);
```

Hàm này sẽ cho phép ta xác định ba yếu tố khi vẽ đường thẳng, đó là : Kiểu đường, bề dày và mẫu tự tạo.

Dạng đường do tham số **kiểu_đường** xác định. Bảng dưới đây cho các giá trị khả dĩ của **kiểu_đường** :

Các giá trị khả dĩ của kiểu_đường

Tên hằng	Giá trị số	Kiểu đường
SOLID_LINE	0	Nét liền
DOTTED_LINE	1	Nét chấm
CENTER_LINE	2	Nét chấm gạch
DASHED_LINE	3	Nét gạch
USERBIT_LINE	4	Mẫu tự tạo

Hàm vẽ đường thẳng

Bề dày của đường vẽ do tham số **độ_dày** xác định,. bảng dưới đây cho các giá trị khả dĩ của **độ_dày** :

các giá trị khả dĩ của độ_dày :

Tên hằng	Giá trị số	Bề dày
NORM_WIDTH	1	Bề dày bình thường
THICK_WIDTH	3	Bề dày gấp ba

Mẫu tự tạo : Nếu tham số thứ nhất là USERBIT_LINE thì ta có thể tạo ra mẫu đường thẳng bằng tham số **mẫu**.

Ví dụ 2

- ❖ Chương trình vẽ một đường gấp khúc bằng các đoạn thẳng. Đường gấp khúc đi qua các đỉnh sau: (20,20),(620,20),(620,180),(20,180) và (320,100)

```
//(20,20),(620,20),(620,180),(20,180) và (320,100)
#include <graphics.h>
#include <stdio.h>
#include <conio.h>
main()
{
    int mh=0, mode;
    initgraph(&mh,&mode,"");
    setbkcolor(BLUE);
    setcolor(YELLOW);
    setlinestyle(SOLID_LINE,0,THICK_WIDTH);
    moveto(320,100); // con chạy ở vị trí ( 320,100 )
    line(20,20,620,20); //con chạy vẫn ở vị trí ( 320,100 )
    linerel(-300,80); //ve qua đỉnh 20, 180
    lineto(620,180);
    lineto(620,20);
    getch();
    closegraph();
}
```

Hàm :

```
void putpixel(int x, int y, int color);
```

sẽ tô điểm (x,y) theo màu xác định bởi **color**.

Hàm

```
unsigned getpixel(int x, int y);
```

sẽ trả về số hiệu màu của điểm ảnh ở vị trí (x,y).

Nếu điểm này chưa được tô màu bởi các hàm vẽ hoặc hàm putpixel (mà chỉ mới được tạo màu nền bởi setbkcolor) thì hàm cho giá trị 0.

Để tô màu cho một miền nào đó trên màn hình, ta dùng hàm sau :

```
void floodfill(int x, int y, int border);
```

ở đây :

(x,y) là toạ độ của một điểm nào đó gọi là điểm gieo.

Tham số border chứa mã của màu.

Sự hoạt động của hàm floodfill phụ thuộc vào giá trị của x,y,border và trạng thái màn hình.

- + Khi trên màn hình có một đường cong khép kín hoặc đường gấp khúc khép kín mà mã màu của nó bằng giá trị của border thì :
 - Nếu điểm gieo (x,y) nằm trong miền này thì miền giới hạn phía trong đường sẽ được tô màu.
 - Nếu điểm gieo (x,y) nằm ngoài miền này thì miền phía ngoài đường sẽ được tô màu.
- + Trong trường hợp khi trên màn hình không có đường cong nào như trên thì cả màn hình sẽ được tô màu.

Ví dụ 3

- ❖ Vẽ một đường tròn màu đỏ trên màn hình màu xanh. Tọa độ (x,y) của điểm gieo được nạp từ bàn phím. Tùy thuộc giá trị cụ thể của x,y chương trình sẽ tô màu vàng cho hình tròn hoặc phần màn hình bên ngoài hình tròn.

```
#include <graphics.h>
#include <stdio.h>
#include <conio.h>
main()
{
    int mh=0, mode, x, y;
    printf("\nNhập vào tọa độ x,y:");
    scanf("%d%d",&x,&y); //tọa độ này để gieo màu trong hay ngoài đường tròn
    initgraph(&mh,&mode,"");

    setbkcolor(BLUE);
    setcolor(RED);
    setfillstyle(11,YELLOW);
    circle(320,100,50);
    moveto(1,150);
    floodfill(x,y,RED);
    getch();
    closegraph();
}
```


Hình chữ nhật

Hàm

```
void rectangle(int x1, int y1, int x2, int y2);
```

sẽ vẽ một hình chữ nhật có các cạnh song song với các cạnh của màn hình. Toạ độ đỉnh trái trên của hình chữ nhật là $(x1, y1)$ và toạ độ đỉnh phải dưới của hình chữ nhật là $(x2, y2)$.

Hàm

```
void bar(int x1, int y1, int x2, int y2);
```

sẽ vẽ và tô màu một hình chữ nhật. Toạ độ đỉnh trái trên của hình chữ nhật là $(x1, y1)$ và toạ độ đỉnh phải dưới của hình chữ nhật là $(x2, y2)$.

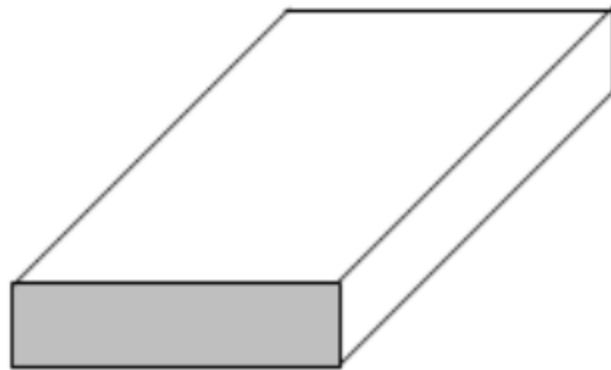
...

Hình chữ nhật

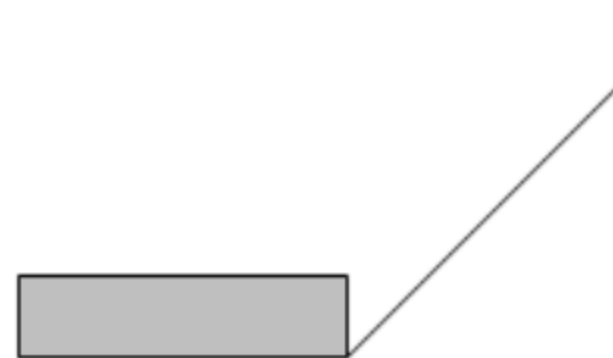
Hàm

```
void bar3d(int x1, int y1, int x2, int y2, int depth, int top);
```

sẽ vẽ một khối hộp chữ nhật, mặt ngoài của nó là hình chữ nhật xác định bởi các tọa độ $(x1, y1)$, $(x2, y2)$. Hình chữ nhật này được tô màu thông qua hàm `setfillstyle`. Tham số **depth** xác định số điểm ảnh trên bề sâu của khối 3 chiều. Tham số **top** có thể nhận các giá trị 1 hay 0 và khối 3 chiều tương ứng sẽ có nắp hoặc không.



`top=1`



`top=0`

Ví dụ 4

- ❖ Chương trình dưới đây tạo nên một hình chữ nhật, một khối hình chữ nhật và một hình hộp có nắp

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>
#include <math.h>

main()
{
    int mh=0,mode;
    initgraph(&mh,&mode,"");
    setbkcolor(GREEN);
    setcolor(RED);
    setfillstyle(CLOSE_DOT_FILL,YELLOW);
    rectangle(5,5,300,160);
    bar(3,175,300,340);
    bar3d(320,100,500,340,100,1);
    getch();
    closegraph();
}
```

Vẽ đa giác

Vẽ đường gấp khúc

Muốn vẽ đường gấp khúc đi qua n điểm : (x_1, y_1) , (x_2, y_2) , ..., (x_n, y_n) thì trước hết ta phải gán các tọa độ (x_i, y_i) cho một mảng a kiểu `int` nào đó theo nguyên tắc sau :

Tọa độ x_1 gán cho $a[0]$

Tọa độ y_1 gán cho $a[1]$

Tọa độ x_2 gán cho $a[2]$

Tọa độ y_2 gán cho $a[3]$

....

Tọa độ x_n gán cho $a[2n-2]$

Tọa độ y_n gán cho $a[2n-1]$

Sau đó gọi hàm :

`drawpoly(n,a);`

Nếu điểm cuối cùng (x_n, y_n) trùng với điểm đầu (x_1, y_1) thì ta nhận được một đường gấp khúc khép kín.

Tô màu đa giác

Giả sử ta có a là mảng đã đề cập đến trong mục trên, khi đó ta gọi hàm :

```
fillpoly(n,a);
```

Ví dụ 5

❖ Vẽ một đường gấp khúc và hai đường tam giác.

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>
#include <math.h>

int poly1[]={5,200,190,5,100,300}; //đường gấp khúc
int poly2[]={205,200,390,5,300,300}; //Tam giác 1
int poly3[]={405,200,590,5,500,300,405,200}; //Tam giác 2

main()
{
    int md=0,mode;
    initgraph(&md,&mode,"");
    setbkcolor(CYAN);
    setcolor(YELLOW);
    setfillstyle(SOLID_FILL,MAGENTA);
    drawpoly(3,poly1);
    fillpoly(3,poly2);
    fillpoly(4,poly3);
    getch();
    closegraph();
}
```